EXPERIMENT – 1

Foundation of Deep Learning

Evolution from Perceptron to Multi-Layer Perceptron (MLP):

Objective:

To understand why the Single-Layer Perceptron (SLP) fails for some classification problems and how the Multi-Layer Perceptron (MLP) overcomes these limitations by learning nonlinear decision boundaries. The experiment begins with the AND and OR logic gate datasets which are linearly separable and then the XOR dataset which is not. This shows the need for hidden layers in neural networks.

Introduction:

Artificial Neural Networks are created based on the working principle of biological neurons. The most basic neural network is the Single-Layer Perceptron (SLP), which contains an input layer and one output neuron. A Single-Layer Perceptron network can classify only linearly separable problems effectively.

In order to comprehend this restriction, let us consider the following three logical gates' data sets:

- AND Gate
- OR Gate
- XOR Gate

The data sets for AND and OR gates can be distinguished using one straight line. Thus, they can be classified using a Single-Layer Perceptron.

But, the data set for XOR gate cannot be distinguished using one straight line, as its classes are located diagonally opposite to each other. This restriction leads to the development of MLP (Multi-Layer Perceptron)..

Task 1: Explain the architecture of a Single-Layer Perceptron

Single Layer Perceptron (SLP)

A Single Layer Perceptron (SLP) is the simplest form of an artificial neural network. An SLP comprises an input layer with connections straight to an output neuron. Each of the inputs will be multiplied by its respective weight and a bias is added to it. The outcome will be fed into an activation function, which can either be the sigmoid or the step function.
The mathematical equation of a perceptron is:

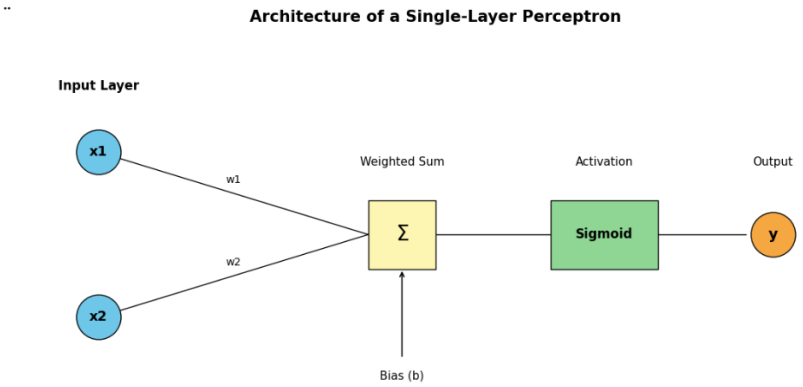
$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

where

- x_i = Input features

- W_i = Weights
- b = Bias
- f = Activation Function

The Single-Layer Perceptron can only classify linearly separable data because it can produce only one linear decision boundary.



Why do we need Multi-Layer Perceptron?

Before studying XOR, let us first examine the AND and OR logical gates.

If a Single-Layer Perceptron can correctly classify all datasets, then there is no need for a Multi-Layer Perceptron.Therefore, we first analyze

- AND
- OR
- XOR

using a Single-Layer Perceptron.

AND Gate:

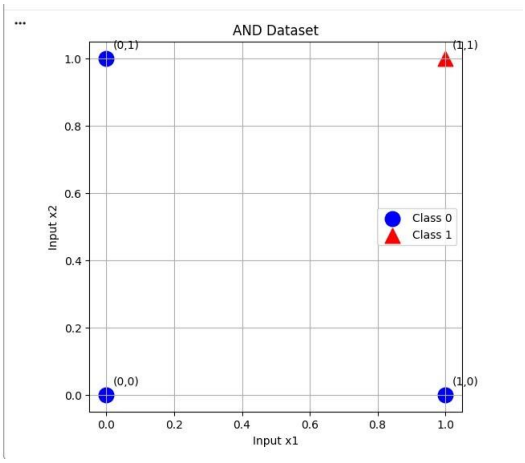
Truth Table:

```
import pandas as pd

and_table=pd.DataFrame({
    "Input x1":[0,0,1,1],
    "Input x2":[0,1,0,1],
    "AND Output":[0,0,0,1]
})

print(and_table)
```

```
   Input x1  Input x2  AND Output
0         0         0           0
1         0         1           0
2         1         0           0
3         1         1           1
```

Plot AND Dataset in 2D Coordinate System:

```
import matplotlib.pyplot as plt
import numpy as np

X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([0,0,0,1])

plt.figure(figsize=(6,6))

plt.scatter(X[y==0][:,0],X[y==0][:,1],
            color="blue",s=180,label="Class 0")

plt.scatter(X[y==1][:,0],X[y==1][:,1],
            color="red",marker="^",s=180,label="Class 1")

labels=["(0,0)","(0,1)","(1,0)","(1,1)"]

for point,label in zip(X,labels):
    plt.text(point[0]+0.02,point[1]+0.03,label)

plt.xlabel("Input x1")
plt.ylabel("Input x2")
plt.title("AND Dataset")
plt.grid(True)
plt.legend()

plt.show()
```



Check Whether the AND Dataset is Linearly Separable:

The AND dataset is linearly separable because a single straight line can separate the output classes.

The points belonging to Class 0 are:

(0,0), (0,1), (1,0)

The point belonging to Class 1 is:

(1,1)

A straight-line decision boundary such as x1+x2=1.5 correctly separates these two classes.

Hence, the AND dataset is linearly separable.

Implement the AND Classifier Using a Single-Layer Perceptron:

```python
import numpy as np
import tensorflow as tf

# AND Dataset
X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
], dtype=np.float32)

y = np.array([
    [0],
    [0],
    [0],
    [1]
], dtype=np.float32)

# Single Layer Perceptron
model = tf.keras.Sequential([
    tf.keras.Input(shape=(2,)),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

# Compile
model.compile(
    optimizer=tf.keras.optimizers.SGD(learning_rate=0.1),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Train
history = model.fit(
    X,
    y,
    epochs=3000,
    verbose=0
)

# Evaluate
loss, accuracy = model.evaluate(X, y, verbose=0)

print(f"\nLoss: {loss:.4f}")
print(f"Accuracy: {accuracy*100:.2f}%")

# Predictions
predictions = model.predict(X, verbose=0)

print("\nPredictions")
print("-"*55)
print("Input\tActual\tProbability\tPredicted")

for i in range(len(X)):
    predicted = 1 if predictions[i][0] >= 0.5 else 0
    print(f"{X[i]}\t{int(y[i][0])}\t{predictions[i][0]:.4f}\t\t{predicted}")
```

```
Loss: 0.0568
Accuracy: 100.00%

Predictions
-------------------------------------------------------
Input   Actual  Probability     Predicted
[0. 0.] 0       0.0005          0
[0. 1.] 0       0.0641          0
[1. 0.] 0       0.0641          0
[1. 1.] 1       0.9100          1
```

Observation

The Single-Layer Perceptron successfully classified all four AND gate input combinations. The predicted probabilities for Class 0 are close to 0, while the probability for Class 1 is close to 1. The model achieved 100% prediction accuracy, indicating successful learning of the AND logical operation.

Conclusion

Since the AND dataset is linearly separable, a Single-Layer Perceptron is sufficient to classify it correctly. Therefore, no hidden layer is required for solving the AND gate problem.

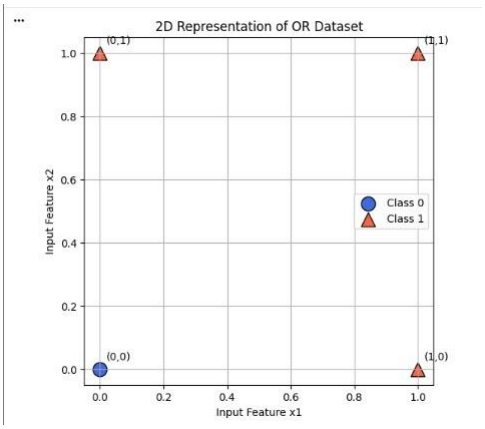
OR Gate:

```
import pandas as pd

or_table = pd.DataFrame({
    "Input x1": [0, 0, 1, 1],
    "Input x2": [0, 1, 0, 1],
    "OR Output": [0, 1, 1, 1]
})

print(or_table)
```

```
   Input x1  Input x2  OR Output
0         0         0          0
1         0         1          1
2         1         0          1
3         1         1          1
```

Plot the OR Dataset in a 2D Coordinate System

```python
import numpy as np
import matplotlib.pyplot as plt

X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

y = np.array([0,1,1,1])

plt.figure(figsize=(6,6))

# Class 0
plt.scatter(
    X[y==0][:,0],
    X[y==0][:,1],
    color="royalblue",
    s=180,
    edgecolor="black",
    label="Class 0"
)

# Class 1
plt.scatter(
    X[y==1][:,0],
    X[y==1][:,1],
    color="tomato",
    marker="^",
    s=180,
    edgecolor="black",
    label="Class 1"
)

labels=["(0,0)","(0,1)","(1,0)","(1,1)"]

for point,label in zip(X,labels):
    plt.text(point[0]+0.02,point[1]+0.03,label)

plt.xlabel("Input Feature x1")
plt.ylabel("Input Feature x2")
plt.title("2D Representation of OR Dataset")
plt.grid(True)
plt.legend()

plt.show()
```



Check Whether OR is Linearly Separable

The OR dataset is linearly separable because a single straight line can correctly separate the two output classes.

The points belonging to Class 0 are (0,0). The points belonging to Class 1 are (0,1), (1,0), (1,1)

A straight-line decision boundary such as x1+x2=0.5 correctly separates the two classes. Therefore, the OR dataset is linearly separable.

Implement OR Classifier Using Single-Layer Perceptron

```python
import numpy as np
import tensorflow as tf

# OR Dataset
X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
], dtype=np.float32)

y = np.array([
    [0],
    [1],
    [1],
    [1]
], dtype=np.float32)

# Single Layer Perceptron
model = tf.keras.Sequential([
    tf.keras.Input(shape=(2,)),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

# Compile
model.compile(
    optimizer=tf.keras.optimizers.SGD(learning_rate=0.1),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Train
history = model.fit(
    X,
    y,
    epochs=3000,
    verbose=0
)

# Evaluate
loss, accuracy = model.evaluate(X, y, verbose=0)

print(f"\nLoss: {loss:.4f}")
print(f"Accuracy: {accuracy*100:.2f}%")

# Predictions
predictions = model.predict(X, verbose=0)

print("\nPredictions")
print("-"*55)
print("Input\tActual\tProbability\tPredicted")

for i in range(len(X)):
    predicted = 1 if predictions[i][0] >= 0.5 else 0
    print(f"{X[i]}\t{int(y[i][0])}\t{predictions[i][0]:.4f}\t\t{predicted}")
```

```
Loss: 0.0321
Accuracy: 100.00%

Predictions
-----------------------------------------------------------
Input   Actual  Probability     Predicted
[0. 0.] 0       0.0697          0
[0. 1.] 1       0.9723          1
[1. 0.] 1       0.9723          1
[1. 1.] 1       0.9999          1
```

Observation

The Single-Layer Perceptron correctly classified all four OR gate input combinations. The model achieved 100% prediction accuracy, showing that it successfully learned the OR logical operation.

Conclusion

Since the OR dataset is linearly separable, it can be correctly classified using a Single-Layer Perceptron. Therefore, no hidden layer is required.

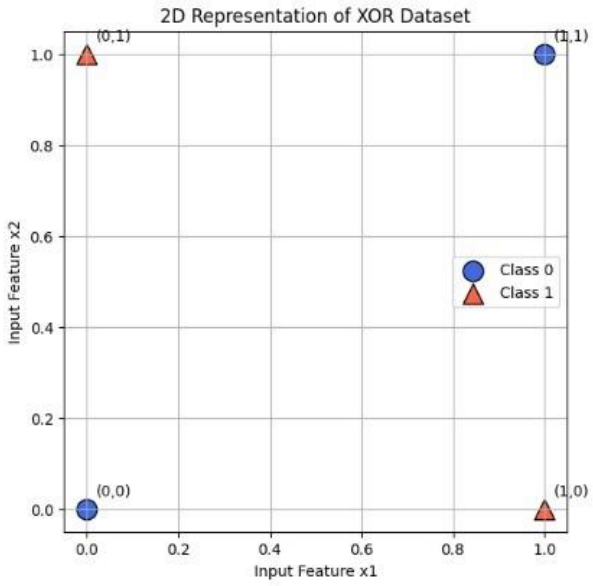
XOR Gate:

```python
import pandas as pd

xor_table = pd.DataFrame({
    "Input x1": [0, 0, 1, 1],
    "Input x2": [0, 1, 0, 1],
    "XOR Output": [0, 1, 1, 0]
})

print(xor_table)
```

```
   Input x1  Input x2  XOR Output
0         0         0           0
1         0         1           1
2         1         0           1
3         1         1           0
```

Plot the XOR Dataset in a 2D Coordinate System:

```python
import numpy as np
import matplotlib.pyplot as plt

X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

y = np.array([0,1,1,0])

plt.figure(figsize=(6,6))

# Class 0
plt.scatter(
    X[y==0][:,0],
    X[y==0][:,1],
    color="royalblue",
    s=180,
    edgecolor="black",
    label="Class 0"
)

# Class 1
plt.scatter(
    X[y==1][:,0],
    X[y==1][:,1],
    color="tomato",
    marker="^",
    s=180,
    edgecolor="black",
    label="Class 1"
)

labels=["(0,0)","(0,1)","(1,0)","(1,1)"]

for point,label in zip(X,labels):
    plt.text(point[0]+0.02,point[1]+0.03,label)

plt.xlabel("Input Feature x1")
plt.ylabel("Input Feature x2")
plt.title("2D Representation of XOR Dataset")
plt.grid(True)
plt.legend()

plt.show()
```



Check Whether XOR is Linearly Separable:

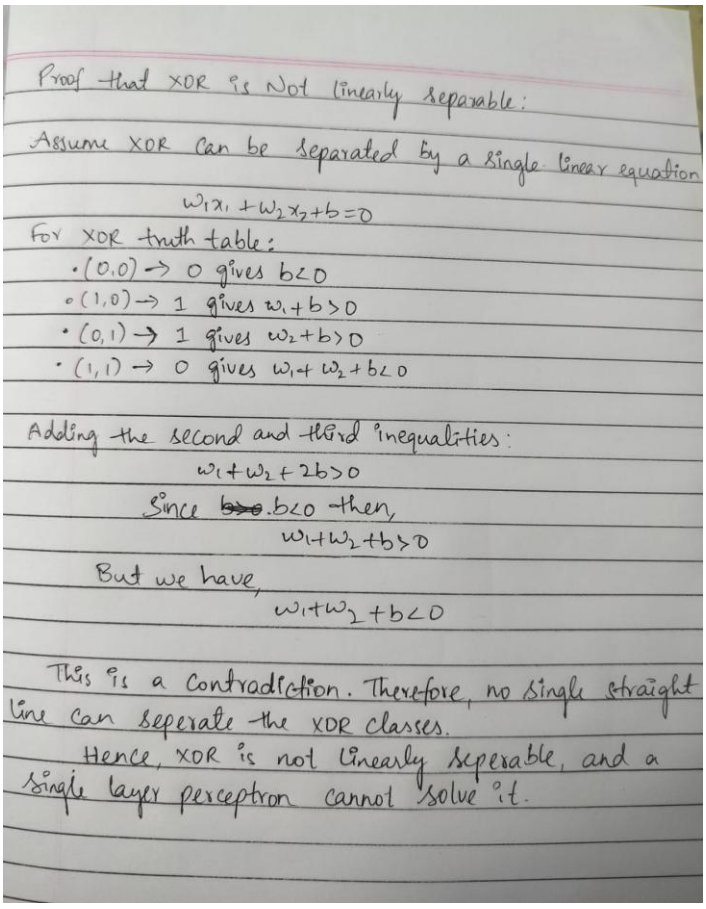
The XOR dataset is not linearly separable because no single straight-line decision boundary can correctly separate the two classes.

The points belonging to Class 0 are: (0,0), (1,1)

The points belonging to Class 1 are: (0,1), (1,0)

These two classes are positioned diagonally opposite each other. Any straight line drawn to separate one Class 0 point will incorrectly classify the other. Hence, no single linear decision boundary can correctly classify all four points.

Therefore, the XOR dataset is not linearly separable.



Why No Single Linear Decision Boundary Can Correctly Classify XOR:

A linear decision boundary is a straight line that separates data points belonging to different classes.

In the XOR dataset:

Class 0 consists of (0,0) and (1,1). Class 1 consists of (0,1) and (1,0).

Since the points belonging to the same class are located diagonally opposite each other, it is impossible to draw a single straight line that separates all Class 0 points from all Class 1 points without making classification errors. Therefore, a Single-Layer Perceptron cannot correctly solve the XOR problem.

Task 6: Discuss How Introducing a Hidden Layer Enables Learning Nonlinear Decision Boundaries

A Single-Layer Perceptron can learn only linear decision boundaries, which means it can classify only linearly separable datasets such as the AND and OR gates. However, the XOR dataset is nonlinearly separable because its data points belonging to the same class are located diagonally opposite each other. As a result, no single straight line can correctly separate the two classes.

A Multi-Layer Perceptron (MLP) overcomes this limitation by introducing one or more hidden layers between the input and output layers.

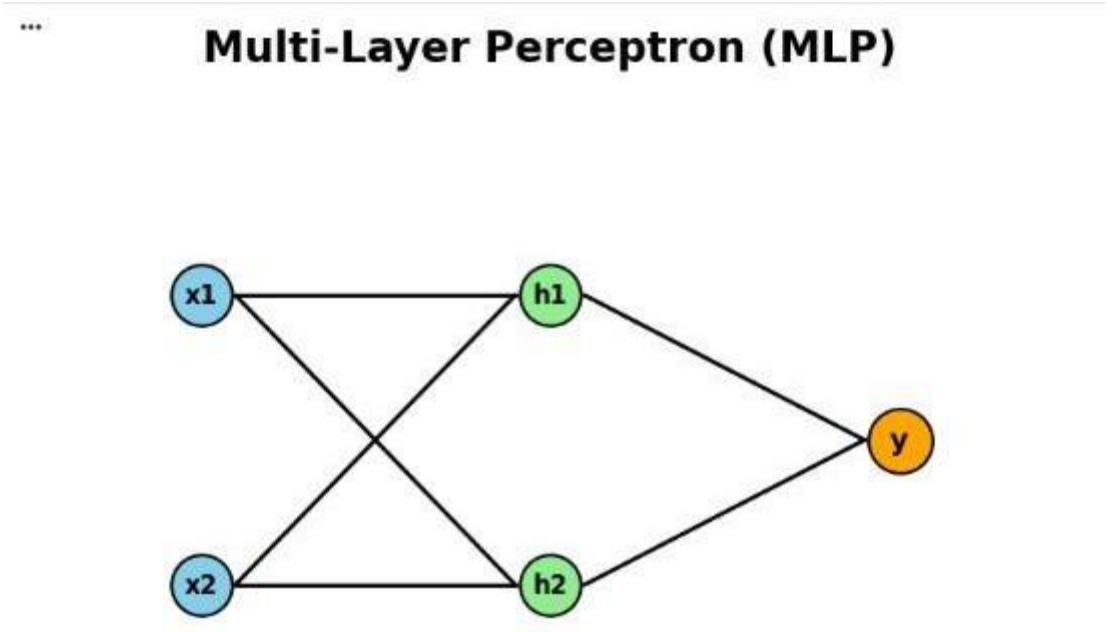
Each hidden neuron learns an intermediate representation of the input. Together, these learned representations enable the network to form nonlinear decision boundaries.

The hidden layer uses nonlinear activation functions such as the sigmoid function, enabling the network to transform the original input space into a new feature space where the XOR classes become separable. Therefore, introducing hidden layers significantly improves the learning capability of neural networks and allows them to solve complex nonlinear classification problems that cannot be solved using a Single-Layer Perceptron.

Task 7: Draw an MLP Architecture Capable of Solving XOR

```python
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8,5))

ax.set_xlim(0,8)
ax.set_ylim(0,6)
ax.axis('off')

# ---------------- Input Layer ----------------
inputs = [(1,4), (1,1)]

for i, (x, y) in enumerate(inputs):
    ax.scatter(x, y,
               s=850,
               color='skyblue',
               edgecolors='black',
               linewidth=1.5)
    ax.text(x, y, f'x{i+1}',
            ha='center',
            va='center',
            fontsize=12,
            fontweight='bold')

# ---------------- Hidden Layer ----------------
hidden = [(4,4), (4,1)]

for i, (x, y) in enumerate(hidden):
    ax.scatter(x, y,
               s=850,
               color='lightgreen',
               edgecolors='black',
               linewidth=1.5)
    ax.text(x, y, f'h{i+1}',
            ha='center',
            va='center',
            fontsize=12,
            fontweight='bold')

# ---------------- Output Layer ----------------
output = (7,2.5)

ax.scatter(output[0], output[1],
           s=950,
           color='orange',
           edgecolors='black',
           linewidth=1.5)

ax.text(output[0], output[1], 'y',
        ha='center',
        va='center',
        fontsize=14,
        fontweight='bold')
```

```
# ---------------- Connections ----------------
# x1 → h1
ax.plot([1.3,3.7],[4,4],color='black',linewidth=2)

# x1 → h2
ax.plot([1.3,3.7],[4,1],color='black',linewidth=2)

# x2 → h1
ax.plot([1.3,3.7],[1,4],color='black',linewidth=2)

# x2 → h2
ax.plot([1.3,3.7],[1,1],color='black',linewidth=2)

# Hidden → Output
ax.plot([4.3,6.7],[4,2.5],color='black',linewidth=2)
ax.plot([4.3,6.7],[1,2.5],color='black',linewidth=2)

# ---------------- Title ----------------
plt.title("Multi-Layer Perceptron (MLP)",
          fontsize=20,
          fontweight='bold',
          pad=20)

plt.show()
```



The MLP consists of:

Input Layer: Two input neurons (x1and x2).

Hidden Layer: Two hidden neurons with sigmoid activation functions.

Output Layer: One output neuron with sigmoid activation.

The hidden layer transforms the input features into a higher-dimensional feature space, allowing the network to learn nonlinear decision boundaries.

Task 8: Implement an XOR Classifier Using TensorFlow with One Hidden Layer, Sigmoid Activation, and Binary Cross Entropy Loss

```
import numpy as np
import tensorflow as tf

# Clear previous session
tf.keras.backend.clear_session()

# Random seed
np.random.seed(42)
tf.random.set_seed(42)

# XOR Dataset
X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
], dtype=np.float32)

y = np.array([
    [0],
    [1],
    [1],
    [0]
], dtype=np.float32)

# Build MLP Model
model = tf.keras.Sequential([
    tf.keras.Input(shape=(2,)),
    tf.keras.layers.Dense(
        4,
        activation='sigmoid'
    ),
    tf.keras.layers.Dense(
        1,
        activation='sigmoid'
    )
])
```

```python
# Evaluate Model
loss, accuracy = model.evaluate(
    X,
    y,
    verbose=0
)

print("Final Model Performance")
print("-"*40)
print(f"Loss     : {loss:.4f}")
print(f"Accuracy : {accuracy*100:.2f}%")

# Predictions
pred = model.predict(X, verbose=0)

print("\nNetwork Predictions")
print("-"*70)
print("Sample\tInput\tTarget\tProbability\tPrediction")

for i in range(len(X)):
    p = pred[i][0]
    c = 1 if p >= 0.5 else 0

    print(f"{i+1}\t{X[i]}\t{int(y[i][0])}\t{p:.4f}\t\t{c}")
```

```
Final Model Performance
----------------------------------------
Loss     : 0.0547
Accuracy : 100.00%
WARNING:tensorflow:5 out of the last 5 calls to <function TensorFlowTrainer.make

Network Predictions
----------------------------------------------------------------------
Sample  Input   Target  Probability     Prediction
1       [0. 0.] 0       0.0150          0
2       [0. 1.] 1       0.9805          1
3       [1. 0.] 1       0.9157          1
4       [1. 1.] 0       0.0917          0
```

The Multi-Layer Perceptron successfully learned the XOR dataset and correctly classified all four input combinations. Unlike the Single-Layer Perceptron, the MLP successfully classified the XOR dataset because the hidden layer transformed the input into a feature space where the classes became separable.

TaskG: Plot the training loss over epochs

```python
import matplotlib.pyplot as plt

plt.figure(figsize=(9,5))

plt.plot(
    history.history['loss'],
    color='green',
    linewidth=3,
    linestyle='--',
    marker='o',
    markersize=3,
    label='Loss'
)

plt.title("MLP XOR Training Loss")
plt.xlabel("Epoch Number")
plt.ylabel("Loss")
```

The trained Multi-Layer Perceptron correctly classified all four XOR input combinations, achieving 100% prediction accuracy. The predicted probabilities were on the correct side of the decision threshold (0.5), resulting in accurate classification of each input. This demonstrates that the hidden layer successfully learned the nonlinear relationship present in the XOR dataset.

Task 10: Report the Final Prediction Accuracy

```python
# Final Predictions

pred = model.predict(X, verbose=0)

print("Final Prediction Results")
print("-"*75)
print("Sample\tInput\tTarget\tProbability\tPredicted Class")

for i in range(len(X)):

    probability = pred[i][0]

    predicted_class = 1 if probability >= 0.5 else 0

    print(f"{i+1}\t{X[i]}\t{int(y[i][0])}\t{probability:.4f}\t\t{predicted_class}")

# Model Evaluation

loss, accuracy = model.evaluate(X, y, verbose=0)

print("\nModel Evaluation")
print("-"*30)
print(f"Final Loss     : {loss:.4f}")
print(f"Final Accuracy : {accuracy*100:.2f}%")
```

```
Final Prediction Results
---------------------------------------------------------------------------
Sample  Input   Target  Probability     Predicted Class
1       [0. 0.] 0       0.0150          0
2       [0. 1.] 1       0.9805          1
3       [1. 0.] 1       0.9157          1
4       [1. 1.] 0       0.0917          0

Model Evaluation
------------------------------
Final Loss     : 0.0547
Final Accuracy : 100.00%
```

Result

The MLP was tested with the XOR data set after the completion of the training process. The accuracy of the predictions made by the model is 100% because all the four XOR data samples have been classified correctly. All the predicted probabilities are either greater than or less than 0.5 threshold value, hence leading to proper classification of all samples. The plot of decision boundaries indicates that the MLP has managed to learn a nonlinear decision boundary that can separate the XOR classes, something impossible with Single Layer Perceptron.